

LINGUAGEM DE PROGRAMAÇÃO C++

Curso de Capacitação em Desenvolvimento Full Stack - MÓDULO 08 (40H)

Elaborado por: Prof(a). Esp. Érika D. G. R. Dos Santos

ESTRUTURAS CONDICIONAIS



AULA 4: LINGUAGEM DE PROGRAMAÇÃO C++



Condicionais if e else

Tomando Decisões no Código

```
if (condicao) {  
    // executa se condicao for verdadeira  
} else if (outraCondicao) {  
    // executa se outraCondicao for verdadeira  
} else {  
    // executa se nenhuma condicao for verdadeira  
}
```

Exemplo: sistema de aprovação (média $\geq 7 \rightarrow$ "Aprovado", senão "Reprovado")

switch-case e Operador Ternário

Alternativas para Condicionais

- switch-case: para múltiplas comparações com valores fixos

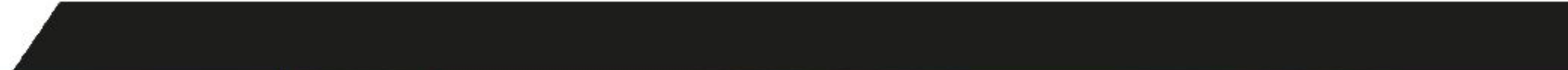
cpp

```
switch (opcao) {  
    case 1: cout << "Opção 1"; break;  
    case 2: cout << "Opção 2"; break;  
    default: cout << "Inválido";  
}
```

- Operador ternário: `(condicao) ? valor_se_verdadeiro : valor_se_falso;`

ESTRUTURAS DE REPETIÇÃO

AULA 5: LINGUAGEM DE PROGRAMAÇÃO C++



Laço for

Repetição com Número Definido de Vezes

```
for (inicializacao; condicao; incremento) {  
    // código executado enquanto condicao for verdadeira  
}
```

Exemplo: tabuada do 5

cpp

```
for (int i = 1; i <= 10; i++) {  
    cout << "5 x " << i << " = " << 5*i << endl;  
}
```

Laços while e do-while

Repetição com Condição no Início ou no Final

- **while**: condição testada no início

```
while (condicao) {  
    // código executado enquanto condicao for verdadeira  
}
```

- **do-while**: condição testada no final (executa pelo menos uma vez)

```
do {  
    // código executado pelo menos uma vez  
} while (condicao);
```

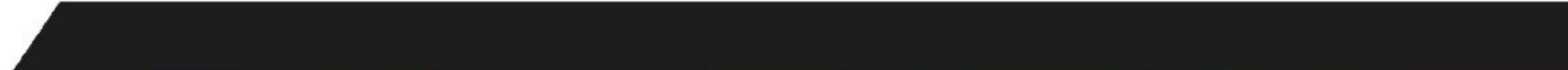

break e continue

Controle de Fluxo em Laços

- **break**: interrompe imediatamente o laço (sai da repetição)
- **continue**: pula o restante do código e vai para a próxima iteração
- Exemplo: validação de senha com tentativas limitadas

EXCEÇÕES E REVISÃO

AULA 6: LINGUAGEM DE PROGRAMAÇÃO C++



Tratamento de Exceções

try, catch, throw

```
try {  
    // código que pode lançar uma exceção  
    if (erro) throw "Mensagem de erro";  
} catch (const char* msg) {  
    // tratamento da exceção  
    cout << msg << endl;  
}
```

Exemplo: divisão por zero, entrada inválida do usuário

Revisão Integrada

Condicionais + Laços + Exceções

- **Exercício prático:** sistema de validação de login com tentativas limitadas

Elabore um sistema de login em C++, com validação do campo login e senha, usando estrutura condicional, estrutura de repetição, tratamento de erro e exceção.

FUNÇÕES

AULA 7: LINGUAGEM DE PROGRAMAÇÃO C++

Definição e Protótipos de Funções

Reutilizando Código

```
// Protótipo (declaração)
```

```
int somar(int a, int b);
```

```
// Definição (implementação)
```

```
int somar(int a, int b) {
```

```
    return a + b;
```

```
}
```

```
// Chamada
```

```
int resultado = somar(5, 3);
```

- Funções evitam repetição de código
- Melhoram organização e manutenção

Passagem de Parâmetros

Valor vs Referência

- Por valor: copia o valor (alterações não afetam o original)

```
void trocar(int a, int b);
```

- Por referência: usa o endereço original (alterações afetam o original)

```
void trocar(int &a, int &b);
```

- **Por que usar referência? Eficiência e capacidade de modificar variáveis externas**

Sobrecarga e Recursividade

Funções com Mesmo Nome e Chamadas a Si Mesmas

- Sobrecarga: várias funções com o mesmo nome, mas parâmetros diferentes

```
int somar(int a, int b);
```

```
double somar(double a, double b);
```

- Recursividade: função que chama a si mesma (ex: fatorial, Fibonacci)